

# Orientation Map — creative-automation-pipeline

*Glance before the call. Each: what it does / the one decision or gotcha. Source of truth is the code.*

---

## **routes/campaign.ts**

POST /generate, /status, /manifest, /compliance, image serving.

**GOTCHA:** /generate is fire-and-forget; returns runId instantly, run dies with the process.

## **workflow/campaignWorkflow.ts**

Declares the 6 steps via .then()...commit().

**GOTCHA:** purely linear, no branch/parallel. Order is the design.

## **workflow/mastra-compatible.ts**

Hand-rolled shim mimicking Mastra createStep/createWorkflow.

**GOTCHA:** API-shape compatible only. In-memory linear runner, no suspend/resume/persistence.

## **lib/runStore.ts**

In-memory Map of run state + cost accumulator + runReport.

**GOTCHA:** ephemeral Map, self-deletes after 1hr. Crash = total loss. Weakest spot.

## **lib/retry.ts**

Exponential backoff + jitter, regex error classification.

**GOTCHA:** only wraps Gemini + Dropbox. Terminal msg hardcoded "Image generation failed" even for Dropbox.

## **steps/loadBrief.ts**

Parse + Zod-validate YAML/JSON brief.

**GOTCHA:** requires min 2 products; brandPalette + prohibitedWords optional.

## **steps/gatherAssets + agents/assetGatherer.ts**

Reuse existing asset or generate via Gemini.

**GOTCHA:** not an LLM agent, just a for-loop. Re-throws on first product failure (no isolation).

## **steps/renderAspectRatios.ts**

Sharp resize to 1x1 / 9x16 / 16x9, cover-fit. Writes base.png.

**GOTCHA:** sequential per product+ratio, no parallelism.

## **steps/applyOverlay.ts**

SVG message bar (auto-shrink font) + optional logo.

Writes final.png.

**GOTCHA:** the black bar it adds later pollutes the color compliance check.

## **steps/organizeOutputs.ts**

Verify files, upload via storage adapter, write manifest.json.

**GOTCHA:** upload failure is soft. Run still "done", falls back to local path.

## **steps/checkCompliance + agents/complianceChecker.ts**

Color + logo + legal-word scan.

**GOTCHA:** per-image try/catch, the only resilient step. Drill the three scorers.

## **tools/extractColors.ts**

50x50 quantized dominant colors, Euclidean palette score, pass  $\geq 0.3$ .

**GOTCHA:** empty palette = score 0 = every image FAILS color compliance.

## **tools/checkLogoPresence.ts**

Claude vision yes/no on any logo/text/watermark.

**GOTCHA:** no reference matching, no retry, errors swallowed to false. Campaign text reads as "logo present."

## **tools/generateAssetWithGenAI.ts**

Claude crafts prompt, Gemini renders image.

**GOTCHA:** image cost hardcoded \$0. The Claude prompt call is NOT retried.

## **lib/ai-clients.ts**

Anthropic client factory + model constants.

**GOTCHA:** TEXT\_MODEL = claude-sonnet-4-5. No temperature set anywhere (non-determinism).

## **lib/paths.ts**

Resolves OUTPUT\_DIR / BRIEFS\_DIR / LOGS\_DIR.

**GOTCHA:** OUTPUT\_DIR is ONE shared dir, not per-run. Concurrent runs clobber each other.

## lib/storage/

Adapter interface. Local + Dropbox live, S3/Azure throw.

**GOTCHA:** Dropbox chosen for creative-team workflow, not tech preference. Have that reason ready.

## campaign-logger.ts / progress-bus.ts

Pino structured logs. EventEmitter feeds runStore.

**GOTCHA:** cost ledger has no per-call latency field. Latency is run-level only.

## lib/db/

Empty Drizzle stub, never imported.

**GOTCHA:** this IS your "Postgres later." Honest deferral, not a lie.

## frontend/src/App.tsx

Polls /status every 750ms until done/error.

**GOTCHA:** polling because Replit proxy buffers SSE. No SSE endpoint exists.

## ONE-BREATH SUMMARY

6 linear Zod-gated steps, in-process and in-memory. Two deterministic compliance gates (color, legal words) plus one advisory vision check (logo). **Strengths:** per-step contracts, structured observability, vendor seams. **Three holes to own first:** (1) ephemeral run state = crash loses everything, (2) shared output dir = concurrency clobber, (3) thin compliance scoring.